

Lab3 实验报告

一、思考题

Thinking 2.7 简单了解并叙述 X86 体系结构中的内存管理机制，比较 X86 和 MIPS 在内存管理上的区别。

核心对比：

特性	X86	MIPS
地址转换	段页式：逻辑地址 → 线性地址 → 物理地址	纯页式：虚拟地址 → 物理地址
TLB管理	硬件自动遍历页表填充，只有缺页才进内核	软件管理，TLB Miss直接触发异常由OS处理
ASID	无，切换CR3时需要刷新TLB	有8位ASID，避免切换进程时全刷TLB
地址空间	32位：3G用户 + 1G内核（常规）	固定分区：kuseg + kseg0/1
页表层级	传统2级 → PAE 3级 → x86_64 4级	3级页表（PDX + PTX + Offset）

关键差异：

- X86有段寄存器（CS/DS/SS/ES/FS/GS），MIPS无分段
- MIPS TLB完全软件控制，X86 TLB硬件维护
- X86页表项有A/D位，硬件自动更新；MIPS需软件处理

Thinking 3.1 请结合 MOS 中的页目录自映射应用解释代码中 `e->env_pgdir[PDX(UVPT)] = PADDR(e->env_pgdir) | PTE_V` 的含义。

代码拆解：

```
e->env_pgdir[PDX(UVPT)] // 页目录第 PDX(UVPT) 项
= PADDR(e->env_pgdir)   // 页目录自身的物理地址
| PTE_V;                // 有效位标记
```

自映射原理：



作用：

1. 用户进程可通过UVPT只读访问自己的页表，无需陷入内核
2. 避免了"页表的页表"递归分配开销
3. 公式： $PTE * user_pagetable = (PTE *) UVPT$

Thinking 3.2 elf_load_seg 以函数指针的形式，接受外部自定义的回调函数 map_page。请你找到与之相关的 data 这一参数在此处的来源，并思考它的作用。没有这个参数可不可以？为什么？

调用链：

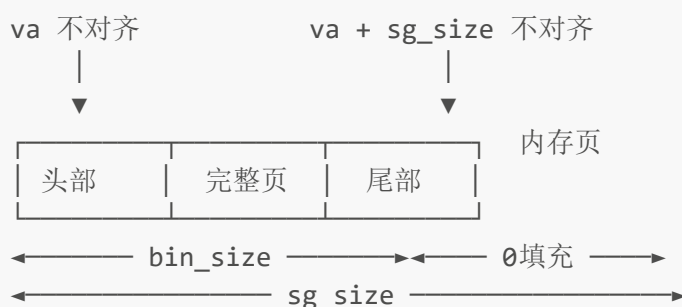
```
load_icode(e, binary, size)
  ↓ 传 e 作为 data
elf_load_seg(ph, bin, load_icode_mapper, e)
  ↓ 回调时透传
load_icode_mapper(data, va, offset, perm, src, len)
  ↓ data 就是 struct Env *e
page_insert(e->env_pgdir, ...)
```

结论：

- **data来源：** load_icode调用elf_load_seg时传入的进程控制块e
- **作用：** 给回调函数传递上下文（当前进程的页目录地址）
- **不能省略：** elf_load_seg是通用ELF解析器，不知道OS具体实现；load_icode_mapper需要知道在哪个进程的地址空间分配页面

设计模式： 这是C语言中实现"闭包"或"上下文绑定"的标准写法

Thinking 3.3 结合 elf_load_seg 的参数和实现，考虑该函数需要处理哪些页面加载的情况。

四种边界情况：

1. **起始不对齐：** va可能不是页对齐，第一个页需从偏移处开始拷贝
2. **尾部剩余：** 文件数据结束后，到页边界的部分用0填充（.bss段）
3. **完整页面：** 中间连续的整页数据直接整页拷贝
4. **权限差异：** 代码段PTE_R | PTE_X，数据段PTE_R | PTE_W

Thinking 3.4 你认为这里的 env_tf.cp0_epc 存储的是物理地址还是虚拟地址?

虚拟地址

理由

1. 用户态执行时, PC始终是虚拟地址, MMU实时翻译
2. 异常发生时, 硬件直接把当前PC值存入EPC, 不做翻译
3. eret返回时, 硬件把EPC值直接写回PC, 也不做翻译
4. 如果存物理地址, 返回时就跳到内核区域了, 用户进程就不知道跳转到哪里了。

Thinking 3.5 试找出0、1、2、3号异常处理函数的具体实现位置。

文件: kern/genex.S

异常号	异常类型	处理函数	定义方式
0	中断	handle_int	NESTED宏直接定义
1	页修改	handle_mod	BUILD_HANDLER(mod, do_mod)
2	TLB Load	handle_tlb	BUILD_HANDLER(tlb, do_tlb_refill)
3	TLB Store	handle_tlb	BUILD_HANDLER(tlb, do_tlb_refill)

BUILD_HANDLER宏展开:

```
NESTED(handle_##exception, TF_SIZE, zero)
    move a0, sp          # 把栈指针 (Trapframe) 传给C函数
    jal handler          # 调用对应的C处理函数
    j    ret_from_exception # 恢复现场返回
END(handle_##exception)
```

Thinking 3.6 阅读 entry.S、genex.S 和 env_asm.S 这几个文件, 并尝试说出时钟中断在哪些时候开启, 在哪些时候关闭。

Status寄存器关键位:

IE	EXL	结果
1	0	中断开启
x	1	中断关闭 (EXL优先级更高)

关闭时机:

1. **异常发生瞬间**：硬件自动置EXL=1，关闭中断
2. **exc_gen_entry**：软件显式清除Status_IE位
3. **上下文保存/恢复**：整个调度过程EXL始终为1

开启时机：

1. **eret执行时**：硬件原子性清EXL=0，IE位生效
2. **进程恢复运行**：RESTORE_ALL恢复Status时，IE=1

用户态运行[中打断] → 时钟中断 → 硬件置EXL=1[中断关]
 → 异常处理 → 调度 → 恢复上下文 → eret[原子清EXL]
 → 用户态运行[中打断]

Thinking 3.7 阅读相关代码，思考操作系统是怎么根据时钟中断切换进程的。

完整流程：

1. 硬件层
Count寄存器自增 → 等于Compare → 置Cause.IP7 → 触发中断
↓
2. 异常入口 (kern/entry.S: exc_gen_entry)
SAVE_ALL → 清Status_IE → 读ExcCode → 查表跳转
↓
3. 中断处理 (kern/genex.S: handle_int)
读Cause & Status → 确认是IM7时钟中断
写0到DEV_RTC_INTERRUPT_ACK 【重要：确认中断】
↓
4. 调度器 (kern/sched.c: schedule)
├ if 需要切换(curenv==NULL 或 时间片用完 或 状态不对 或 yield==1)
│ ├ if 当前进程就绪 → TAILQ_INSERT_TAIL
│ ├ 取下一个进程 TAILQ_FIRST
│ └ count = 新进程优先级
└ else
 └ count--
↓
5. 进程切换 (kern/env.c: env_run)
├ 保存旧上下文到curenv->env_tf (如果有)
├ curenv = 新进程
├ cur_pgdir = 新进程页目录
└ env_pop_tf 恢复上下文 + eret

时间片计算：

- 进程优先级 = N个时钟中断周期
- 每次时钟中断count--，到0切换
- 高优先级进程一次分配更多时间片

二、难点分析与实验体会

1. 核心难点

难点	易错点	解决关键
env_setup_vm地址空间布局	base_pgdir拷贝范围	对照mmu.h的内存布局图
Status寄存器位设置	EXL/UM/IE三者关系	画时序图理解eret的原子性
自映射机制	用物理地址还是虚拟地址	页表项永远存物理地址
env_init链表顺序	倒序插入还是正序	测试用例期望小env_id先分配
schedule调度逻辑	count变量的维护	静态变量只初始化一次

2. 实验收获

- 进程 = PCB + 地址空间 + 执行上下文，三者缺一不可
- 中断是多任务的驱动力，没有时钟中断就没有抢占式调度
- ASID是TLB性能优化的关键，空间换时间
- 回调函数是软件分层的利器，ELF解析器和OS内核完美解耦
- noreturn函数要用j不用jal，否则栈会越来越深

3. 实验注意事项

1. `env_init` 必须倒序插入空闲链表，否则env_id不对，测试不过
2. 自映射那行必须用`PADDR()`，用虚拟地址会引发TLB Miss死循环
3. `cp0_status` 四个位缺一不可：`STATUS_IM7` | `STATUS_IE` | `STATUS_EXL` | `STATUS_UM`
4. 时钟中断处理完必须写ACK到设备，否则下一次中断不会来
5. `TAILQ_REMOVE` 和 `TAILQ_INSERT_TAIL` 配对使用，不要漏删调度队列、
6. 课下需要仔细检查代码，边界越界错误，即使lab完全正确，也要再次核查!!! 如：

```
for(int i = NENV - 1; i >= 0; --i)
{
    envs[i].env_status = ENV_FREE;
    LIST_INSERT_HEAD(&env_free_list, &envs[i], env_link);
}
```

三、原创说明

本报告基于实验指导书和代码实现完成。